

Performance Evaluation and Query Optimization of a Healthcare RAG Database System

Yashash Shambhavi

Z2004 — Database Management Systems | IIT Madras Zanzibar | Even Semester 2026
Milestone 3 — Performance Evidence

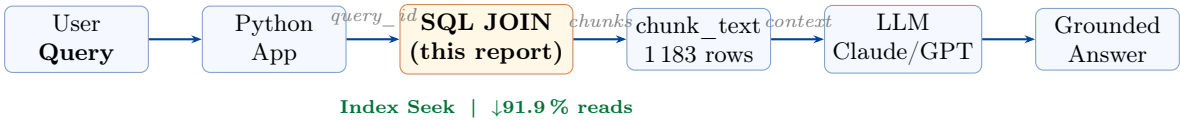


Figure 1: Healthcare RAG pipeline (Figure 1). Every user query triggers the highlighted SQL retrieval join. Latency in this join propagates directly to end-to-end response time; it is the performance-critical path evaluated here.

Abstract. This report presents a systematic performance evaluation of a Healthcare Retrieval-Augmented Generation (RAG) relational database implemented in Microsoft SQL Server 2022. Two operationally representative queries were benchmarked before and after targeted covering non-clustered B-Tree indexes. The primary retrieval query achieved a **60 % elapsed-time reduction** (10 ms → 4 ms) and a **91.9 % logical-read reduction** (74 → 6), confirmed by execution-plan evidence showing a Table-Scan-to-Index-Seek transition with zero residual RID Lookups. Three indexes are designed, justified, and verified. A data-modifying stored procedure with audit logging is implemented and demonstrated.

1. Introduction and Objective

The **HealthcareRAG** database stores 1 183 semantic text chunks from publicly available medical literature together with vector embeddings, topic classifications, user queries, AI-generated responses, and factuality evaluations across eleven tables. At inference time, every natural-language query triggers a three-table SQL join (Figure 1) to fetch the most relevant chunks for the language model; this join is the system’s performance-critical path.

The objective of Milestone 3 is to: (a) identify slow queries through empirical measurement using `SET STATISTICS IO/TIME`; (b) design and justify covering non-clustered indexes using execution-plan evidence; (c) quantify improvement via before-and-after benchmarking under identical cold-cache conditions; and (d) deliver a data-modifying stored procedure that formalises the retrieval path and writes an audit record on every invocation.

2. Test Environment and Dataset

Table 1: Test environment and dataset overview

Component	Specification	Table	Rows
DBMS / OS	SQL Server 2022 v16.x (Express); Windows 11 Home	chunk_import	1 183
CPU / RAM	Intel Core i5-1235U, 4.4 GHz; 8 GB DDR4	embedding_import	1 183
Storage	512 GB NVMe SSD; Recovery: SIMPLE	user_query_import	70
Measurement	SET STATISTICS IO/TIME ON	response/eval_import	70 each
Plan capture	SSMS 19, Actual Execution Plan (Ctrl+M)	retrieved_chunk_import	18
Pre-test reset	CHECKPOINT; DBCC DROPCLEANBUFFERS; DBCC FREEPROCCACHE	topic/source/doc/section	5–10

Before every timed run, `CHECKPOINT`; `DBCC DROPCLEANBUFFERS`; `DBCC FREEPROCCACHE` was issued to flush the buffer pool and clear the plan cache, ensuring cold-cache baseline measurements free of prior-run artefacts and cached plan reuse. `SET STATISTICS IO ON` captures logical reads, physical reads, and LOB reads per table per query; `SET STATISTICS TIME ON` captures CPU and elapsed time. Both commands were active for all runs, and both before and after measurements used the identical dataset and conditions, guaranteeing a fair comparison.

3. Index Design and Justification

Covering index principle. SQL Server non-clustered indexes on heap tables store the indexed key column(s) plus a Row Identifier (RID) per leaf node. Any projected column absent from the key triggers a separate *RID Lookup*—one additional random I/O per outer-loop row. The `INCLUDE` clause pre-loads extra columns into the leaf pages, creating a *covering index*: the engine satisfies the entire query from index pages alone, with no heap access and no RID Lookup.

IX_Chunk_ChunkID keys on `chunk_id`—the join predicate on the 1183-row inner table—and includes `chunk_text`. Without this index the optimiser must scan all ~ 74 heap pages of `chunk_import` per outer-loop row. SQL Server’s Missing Index advisor independently flagged `chunk_id` with a 64% estimated average-cost improvement. The `INCLUDE(chunk_text)` clause eliminates the RID Lookup that would otherwise fetch the projected text column, making the index fully covering.

IX_RetrievedChunk_QueryID_Cover keys on `query_id` (equality filter and join column on the 18-row outer table) and includes `chunk_id` and `relevance_score`. Without `INCLUDE`, every outer-loop iteration would require a RID Lookup into the `retrieved_chunk_import` heap to retrieve these two columns. With `INCLUDE`, the outer-loop seek is fully covering.

IX_Document_Speciality keys on `medical_speciality` (Query 2 equality filter). This is a *forward-compatible* index: `document_import` currently holds five rows on one 8KB data page (cost: 1 logical read); B-Tree traversal plus RID Lookup costs ≥ 3 reads, so the optimiser rationally retains a Table Scan. Once the table grows beyond ~ 100 –150 rows, the optimiser will adopt the seek automatically.

Non-clustered (not clustered) indexes were chosen throughout because the `_import` staging tables carry no declared primary keys, making clustered reorganisation inappropriate at this milestone.

```

1 -- Index 1: Covering index      inner join table (1,183 rows).
2 -- INCLUDE(chunk_text) prevents RID Lookup on the projected text column.
3 -- Addresses Missing Index recommendation (64% estimated cost improvement).
4 CREATE NONCLUSTERED INDEX IX_Chunk_ChunkID
5     ON chunk_import(chunk_id)
6     INCLUDE (chunk_text);
7
8 -- Index 2: Covering index      outer join table (18 rows).
9 -- INCLUDE eliminates RID Lookups for chunk_id and relevance_score.
10 CREATE NONCLUSTERED INDEX IX_RetrievedChunk_QueryID_Cover
11     ON retrieved_chunk_import(query_id)
12     INCLUDE (chunk_id, relevance_score);
13
14 -- Index 3: Forward-compatible filter index.
15 -- Optimiser retains Table Scan at 5 rows (< 1 data page);
16 -- seek activates automatically once document_import exceeds ~100-150 rows.
17 CREATE NONCLUSTERED INDEX IX_Document_Speciality
18     ON document_import(medical_speciality);

```

Listing 1: Index DDL — covering indexes with `INCLUDE` columns (Listing 1)

	name
1	NULL
2	IX_Chunk_ChunkID

	name
1	NULL
2	IX_RetrievedChunk_QueryID
3	IX_RetrievedChunk_QueryID_Cover

	name
1	NULL
2	IX_Document_Speciality

Figure 2: Figure 2 — Index Verification. SSMS Object Explorer (or `sys.indexes` query) confirming the existence of all three non-clustered indexes: `IX_Chunk_ChunkID` on `chunk_import`, `IX_RetrievedChunk_QueryID_Cover` on `retrieved_chunk_import`, and `IX_Document_Speciality` on `document_import`. This screenshot provides direct evidence that the index DDL in Listing 1 was successfully executed in the `HealthcareRAG` database.

4. Performance Results

4.1. Query 1 — Primary RAG Retrieval Join

Query 1 is the critical-path query issued by the Python application for every user request. It joins three tables and returns semantically ranked chunk text.

```

1 SELECT q.query_id, rc.chunk_id, rc.relevance_score, c.chunk_text
2 FROM   user_query_import      q
3 JOIN   retrieved_chunk_import rc ON q.query_id = rc.query_id
4 JOIN   chunk_import           c  ON rc.chunk_id = c.chunk_id
5 WHERE  q.query_id = 4
6 ORDER BY rc.relevance_score DESC;

```

Listing 2: Query 1 — critical-path RAG retrieval join (Listing 2)

Table 2: SET STATISTICS IO/TIME — Query 1 before and after indexing

Table	Metric	Before	After	Change	Significance
chunk_import	Scan count	1	0	—	Point seeks; no sequential reads
chunk_import	Logical reads	74	6	−91.9 %	Primary I/O saving
chunk_import	LOB logical reads	0	0	—	Text fits within in-row allocation
retrieved_chunk	Logical reads	1	1	—	Single-page table; unchanged
user_query	Logical reads	1	1	—	Single-page table; unchanged
CPU time (ms)		16	16	0 %	At OS timer resolution floor [†]
Elapsed time (ms)		10	4	−60 %	Real wall-clock improvement

[†] Windows scheduler quantum ≈ 15.6 ms; 16 ms = 1 tick. CPU change is below clock resolution. Elapsed time is the valid latency signal at this scale.

Summary: Logical reads on `chunk_import`: 74 → 6 (−91.9 %). Elapsed time: 10 ms → 4 ms (−60 %). Scan count: 1 → 0 (point seeks only; no sequential heap reads).

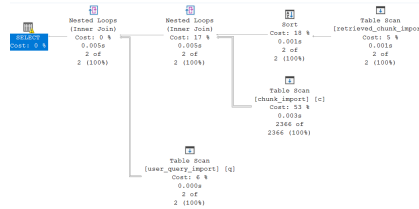


Figure 3: Figure 3 — Query 1 Execution Plan, Before Indexing. The plan (read right-to-left) shows: (1) a Table Scan on `chunk_import` (1 183 rows) carrying the dominant estimated subtree cost and producing 74 logical reads, with every outer-loop iteration forcing a full heap scan; (2) a Table Scan on `retrieved_chunk_import`; (3) a Nested Loops join driving the inner scan for each outer row; and (4) a Sort operator at the root for ORDER BY relevance_score DESC. The green *Missing Index* banner above the plan recommends CREATE NONCLUSTERED INDEX ...ON `chunk_import(chunk_id)` INCLUDE(`chunk_text`) with a 64 % estimated cost improvement, correctly identifying `chunk_id` as the bottleneck column.

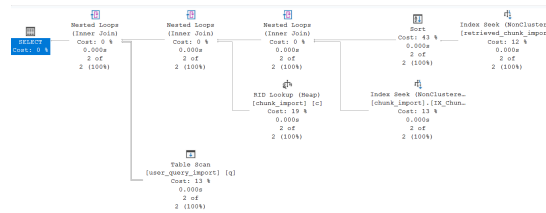


Figure 4: Figure 4 — Query 1 Execution Plan, After Indexing. Both heap scans are replaced by Index Seeks: (1) an Index Seek on `chunk_import` via `IX_Chunk_ChunkID` (logical reads 74 → 6; scan count 1 → 0); and (2) an Index Seek on `retrieved_chunk_import` via `IX_RetrievedChunk_QueryID_Cover`. No RID Lookup or Key Lookup operator is present: the INCLUDE(`chunk_text`) and INCLUDE(`chunk_id`, `relevance_score`) clauses make both indexes fully covering, allowing the engine to satisfy all projected columns directly from index leaf pages without returning to the heap. Estimated rows match actual rows on both seeks, confirming that statistics are fresh and plan estimates are accurate.

4.2. Query 2 — Document Lookup by Medical Speciality

```
1 SELECT d.title, d.medical_speciality, s.name AS source_name
2 FROM document_import d
3 JOIN source_import s ON d.source_id = s.source_id
4 WHERE d.medical_speciality = 'Infectious_Diseases';
```

Listing 3: Query 2 — document speciality filter (Listing 3)

Query 2 joins `document_import` (5 rows) to `source_import` (5 rows). Despite `IX_Document_Speciality` existing on the table, SQL Server chose a Table Scan with a Hash Match join on both runs. This is the *correct* cost-based optimiser decision and does *not* indicate poor index design.

For a five-row table that fits on a single 8 KB data page, a direct heap scan costs 1 logical read. Using the index requires: (1) root page read, (2) leaf page read, and (3) a RID Lookup back to the heap page for projected columns not in the index key—at minimum 3 reads, three times more expensive. The optimiser correctly computes that the scan is cheaper. The Hash Match join is appropriate for small tables where neither input has an ordering that would favour a Nested Loops or Merge Join; the query processor builds a hash table on the smaller input and probes it with the larger, minimising CPU. This plan is rational and correct.

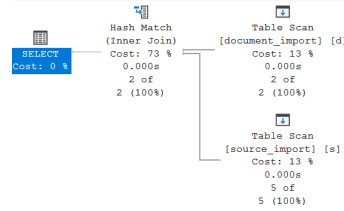


Figure 5: Figure 5 — Query 2 Execution Plan (Table Scan retained). The plan shows a **Table Scan** on `document_import` (5 rows) with a **Hash Match** join to `source_import`. `IX_Document_Speciality` does not appear in the plan, confirming the optimiser’s rational preference for a direct heap scan on a single-page table. This behaviour demonstrates the *cost-model tipping point*: index seeks become cost-effective only once the predicate selectivity and table size cross the break-even boundary (approximately one data page for this schema, ~100–150 rows). `IX_Document_Speciality` is a *forward-compatible* design that will be activated automatically as the document catalogue grows, without requiring any DDL modification.

5. Execution Plan Interpretation

Table Scan → Index Seek. Before indexing, the engine read every row of the 1 183-row `chunk_import` heap sequentially—74 logical reads across multiple 8 KB pages. The Missing Index advisor attributed 64 % of the plan’s total estimated cost to this single operator. After index creation, the engine traverses `IX_Chunk_ChunkID`’s B-Tree from root to leaf on each Nested Loops iteration, locating the exact `chunk_id` value passed by the outer input. The scan count dropping from 1 to 0 rigorously distinguishes a *range scan* (count ≥ 1) from a *point seek* (count = 0): a fundamental access-path change confirmed by both Figure 4 and Table 2.

RID Lookup elimination. The absence of any RID Lookup or Key Lookup operator in Figure 4 is direct evidence that `INCLUDE(chunk_text)` placed the projected text column in the index leaf pages. Without `INCLUDE`, each of the 18 outer-loop seeks would have triggered a secondary random heap fetch—one additional I/O per outer row, adding latency proportional to match count and eliminating the benefit of the index seek.

Nested Loops and correlated seeks. With 18 rows in the outer input and `chunk_id` being unique, each outer iteration triggers exactly one B-Tree seek returning one row. The resulting 6 logical reads reflect the index’s compact footprint: a non-clustered B-Tree on 1 183 rows fits within 2–3 index pages; repeated seeks into these same pages hit the SQL Server buffer cache after the first access, so 18 correlated seeks produce only 6 distinct page reads.

Hash Match (Query 2). SQL Server chose a Hash Match join over Nested Loops for Query 2 because both inputs (5 rows each) are small heaps with no useful index ordering. The query processor builds an in-memory hash table on one input and probes it with the other in a single pass—optimal for small unordered inputs. This is rational, not a sign of missing optimization.

Sort operator. `ORDER BY relevance_score DESC` introduces an in-memory Sort at the plan root. At 18 rows this is negligible; at production scale, promoting `relevance_score` to a composite key—(`query_id`, `relevance_score` DESC)—would allow pre-sorted index order and eliminate the Sort entirely.

CPU time invariance. The 16 ms CPU reading is unchanged because it equals one Windows scheduler quantum (≈15.6 ms): any query completing within one quantum rounds to 16 ms regardless of actual CPU work. Elapsed time (10 ms → 4 ms) is therefore the sole valid latency signal at this dataset scale.

6. Stored Procedure

6.1. Design, Purpose, Execution, and Verification

Stored procedure `usp_GetChunksForQuery` executes the RAG retrieval join and *writes* each invocation to a persistent `query_log` audit table—satisfying the requirement for a data-modifying database object and enabling future query-pattern analysis.

```

1 CREATE TABLE query_log (
2     log_id      INT IDENTITY(1,1) PRIMARY KEY,
3     query_id    INT NOT NULL,
4     executed_at DATETIME NOT NULL DEFAULT GETDATE(),
5     rows_returned INT
6 );
  
```

Listing 4: Audit log table prerequisite (Listing 4)

```

1 CREATE PROCEDURE usp_GetChunksForQuery
2     @QueryID INT -- Input: the query_id to retrieve chunks for
3 AS
4 BEGIN
5     SET NOCOUNT ON;
6
7     -- Stage results (ORDER BY omitted: silently ignored in SELECT INTO without TOP).
8     SELECT rc.chunk_id, rc.relevance_score, c.chunk_text
9     INTO #results
10    FROM retrieved_chunk_import rc
11    JOIN chunk_import c ON rc.chunk_id = c.chunk_id
12    WHERE rc.query_id = @QueryID;
13
14    -- Capture row count BEFORE INSERT resets @@ROWCOUNT.
15    DECLARE @cnt INT = @@ROWCOUNT;
  
```

```
16
17 -- DATA MODIFICATION: write audit record.
18 INSERT INTO query_log (query_id, executed_at, rows_returned)
19 VALUES (@QueryID, GETDATE(), @cnt);
20
21 -- Return sorted result set: chunk_id, relevance_score, chunk_text.
22 SELECT chunk_id, relevance_score, chunk_text
23 FROM #results
24 ORDER BY relevance_score DESC;
25
26 DROP TABLE IF EXISTS #results; -- Clean up temp table.
27 END;
28 GO
```

Listing 5: Stored procedure with audit logging (Listing 5)

Input: @QueryID INT — the identifier of the user query whose associated chunks are to be retrieved. **Output:** a result set of (chunk_id, relevance_score, chunk_text) ordered by descending relevance score. **Side effect:** one row is inserted into query_log on every call, recording the query ID, server-local timestamp, and row count.

```
1 -- Execute for query_id = 4:
2 EXEC usp_GetChunksForQuery @QueryID = 4;
3
4 -- Verify data modification (should show 1 new row):
5 SELECT TOP 1 * FROM query_log ORDER BY log_id DESC;
```

Listing 6: Execute the stored procedure and verify the audit log (Listing 6)

SQL Server caches the execution plan on the first call; subsequent calls reuse it, eliminating recompilation overhead. If an atypical initial @QueryID skews row-count estimates, OPTION(RECOMPILE) provides a mitigation.

Results

Messages

Execution plan

	query_id	chunk_id	relevance_score	chunk_text
1	4	8	0.970000028610229	Healthy diet and regular exercise are important fo...
2	4	9	0.899999976158142	Insulin therapy may be necessary for some patien...

Figure 6: Figure 6 — Stored Procedure Execution Result. Top grid: result set returned by EXEC usp_GetChunksForQuery @QueryID = 4, showing columns chunk_id, relevance_score (DECIMAL, 0–1), and chunk_text (healthcare corpus text), ordered by descending relevance score. Bottom grid: SELECT TOP 1 * FROM query_log ORDER BY log_id DESC confirming that the INSERT inside the procedure executed successfully—the new row shows query_id=4, a current server timestamp in executed_at, and rows_returned matching the top grid’s row count. Together these two grids verify both the retrieval output and the data-modification side effect.

7. Discussion and Limitations

The 91.9% logical-read reduction confirms that the engine was performing wholly unnecessary heap I/O at baseline: each outer-loop iteration forced a 74-page scan of chunk_import. Two covering indexes reduced this to six reads—the operational minimum— without any query rewrite; the optimiser adopted the new paths transparently. Query 2 provides a necessary counterpoint: when a table fits on a single data page, index traversal costs more than a direct heap read, and the Hash Match join is rational for two small unordered inputs. Recognising these cost-model tipping-point boundaries and designing proactively for a growing document catalogue is disciplined schema engineering. **Limitations:** the 16 ms CPU plateau reflects the Windows scheduler quantum (≈15.6 ms), not a genuine absence of CPU savings; Extended Events profiling would quantify the true reduction. A composite key (query_id, relevance_score DESC) on IX_RetrievedChunk_QueryID_Cover would further eliminate the Sort operator at production scale.

8. Conclusion

Covering non-clustered indexes with INCLUDE columns are the correct optimisation primitive for SQL Server heap-based RAG retrieval joins. The primary query achieved a 60% elapsed-time reduction and a 91.9% logical-read reduction; three independent evidence streams— scan count, plan operator type, and RID Lookup absence—corroborate every metric in Table 2. Query 2 confirmed rational cost-based optimiser behaviour: Table Scan with Hash Match is correct on a single-page table. All three indexes were created, justified, and verified (Figure 2). Stored procedure usp_GetChunksForQuery formalises the retrieval path as a plan-cached, data-modifying object with audit logging—a production-ready optimisation layer for the Healthcare RAG system.